

AISSMS

COLLEGE OF ENGINEERING

An Autonomous Institute Affiliated to Savitribai Phule Pune University
Approved by AICTE, New Delhi and Recognised by Govt. of Maharashtra
Accredited by NAAC with “A+” Grade | NBA - 7 UG Programmes

A Mini Project Report On

MediMind Clinical AI Suite

Submitted in partial fulfilment of the requirements for the degree of

THIRD YEAR OF ENGINEERING

In

COMPUTER ENGINEERING

Submitted by

Ansh Devkate – 24CO024

Purvi Kalhapure – 24CO054

Nishigandha Ingle – 24CO041

Under the Guidance of

Prof. M. G. Ghodekar

Department Of Computer Engineering

ALL INDIA SHRI SHIVAJI MEMORIAL SOCIETY’S

COLLEGE OF ENGINEERING Pune – 411001

Academic Year: 2026-27 (Term – I)

Savitribai Phule Pune University

AISSMS
COLLEGE OF ENGINEERING
DEPARTMENT OF COMPUTER ENGINEERING

CERTIFICATE

This is to certify that **Ansh Devkate (24CO024)**, **Purvi Kalhapure (24CO054)** and **Nishigandha Ingle (24CO041)** from Third Year Computer Engineering have successfully completed their mini project work titled “**MediMind Clinical AI Suite**” at AISSMS College of Engineering, Pune in partial fulfilment of the requirements for the degree of Bachelor of Engineering in Computer Engineering.

Prof. M. G. Ghodekar
Project Guide

Dr. D. P. Gaikwad
HOD

Dr. D. S. Bormane
Principal, AISSMS COE

MINI PROJECT APPROVAL

The Mini Project entitled

MediMind Clinical AI Suite

By

Ansh Devkate – 24CO024

Purvi Kalhapure – 24CO054

Nishigandha Ingle – 24CO041

Is approved for the degree of

Third Year of Engineering

In

Computer Engineering

Examiner 1: _____

Name and Signature

Examiner 2: _____

Name and Signature

Date: 23rd September 2026

Place: Pune

ABSTRACT

The MediMind Clinical AI Suite is an advanced, rule-based expert diagnostic and clinical decision support system (CDSS) designed to assist medical practitioners, triage nurses, and students in automated disease diagnosis, symptom-driven inference, and patient vital monitoring. Traditional clinical diagnostic workflows often rely on manual observation across fragmented reference manuals, medical history records, laboratory tests, and differential diagnosis tables. This manual process can be time-consuming, prone to human error under high casualty pressure, and may lead to delayed medical interventions. The proposed MediMind system integrates these diagnostic workflows into a unified, interactive software suite.

The application collects structured patient clinical data, including observed symptoms across multiple body systems (systemic, respiratory, gastrointestinal, and neuro-muscular), physiological vitals (blood pressure, heart rate, body temperature, and oxygen saturation SpO₂), and patient risk factors. This information is validated and held in working memory. The core inference engine executes both Forward Chaining (data-driven reasoning from symptoms to disease diagnosis) and Backward Chaining (goal-driven hypothesis verification from a suspected disease to the required symptom evidence). The system evaluates disease probability scores, generates differential diagnoses, calculates patient triage acuity levels, and produces comprehensive Electronic Health Record (EHR) diagnostic reports.

MediMind is designed as more than a basic symptom checker. It features an interactive Knowledge Base (KB) Editor that allows medical experts to add, modify, or audit production rules (IF-THEN clauses), adjust certainty factors, and update clinical recommendation guidelines. The system retains complete working memory context, enabling clinicians to perform iterative diagnostic updates without re-entering patient data.

The suite is delivered as a full-stack web application: a React 19 + Vite client with ten interactive AI visualizers, and a Node.js + Express backend persisting academic content in MongoDB through Mongoose. The project demonstrates the practical implementation of artificial intelligence, knowledge representation, production systems, and inference algorithms in modern healthcare, and can be extended with IoT vital-sign sensors, EMR integration, and medical imaging classification in the future.

ACKNOWLEDGEMENT

We express our sincere gratitude to our project guide, Prof. M. G. Ghodekar, for her valuable guidance, encouragement, and continuous support throughout the development of this mini project. Her insightful suggestions helped us understand the practical aspects of designing rule-based expert systems, knowledge representation, inference engines, and preparing this academic report.

We sincerely acknowledge the dedicated contributions, cooperation, and consistent efforts of all our team members: Ansh Devkate, Purvi Kalhapure, and Nishigandha Ingle. The successful completion of this project was possible because of the active participation, shared responsibility, and effective teamwork of every member.

We are deeply thankful to Dr. D. P. Gaikwad, Head of the Department of Computer Engineering, and Dr. D. S. Bormane, Principal of AISSMS College of Engineering, Pune, for providing us with the necessary departmental facilities, laboratory resources, and encouraging academic environment required to complete this work.

We also extend our thanks to all faculty and staff members of the Department of Computer Engineering for their direct and indirect support, and to our classmates and friends for their constructive feedback during the testing and refinement of the system.

Finally, we express our heartfelt gratitude to our families for their constant encouragement, patience, and support throughout the completion of this engineering mini project.

Academic Year: 2026-27 (Term – I)

Date: 23rd September 2026

TABLE OF CONTENTS

Sr. No.	Chapter / Section	Page No.
1	Abstract	1
2	Acknowledgement	2
3	Table of Contents	3
4	Introduction	4 - 5
5	Problem Statement and Objectives	6 - 7
6	Software Requirement Specification (SRS)	8 - 10
7	System Analysis and Design	11 - 13
8	AI Based Clinical Diagnostic Systems & Rule Inference	14 - 17
9	Knowledge Base & Working Memory Management	18 - 21
10	API Integration and Backend Implementation	22 - 25
11	Graphical User Interface & Clinical Visualizer	26 - 27
12	System Implementation & Testing	28 - 31
13	Conclusion & Future Scope	32
14	References	33

CHAPTER 1: INTRODUCTION

1.1 Overview of the Project

In modern healthcare systems, rapid and accurate clinical decision-making is vital for saving lives, optimizing hospital triage, and preventing diagnostic errors. Diagnostic decision-making requires analyzing a complex web of symptoms, physiological vitals, patient medical history, and risk factors. Traditional clinical workflows rely heavily on the manual expertise of physicians and nurses, who cross-reference patient symptoms against extensive clinical guidelines and medical literature. Under high patient volumes, such as emergency department triage, this process can become overloaded, leading to delayed interventions or oversight of critical disease markers.

The MediMind Clinical AI Suite is designed to address these challenges by providing an interactive, rule-based expert diagnostic system. Built around production rules (IF-THEN logic) inspired by historic expert systems such as Stanford's MYCIN, MediMind combines data-driven Forward Chaining inference and hypothesis-driven Backward Chaining inference. The system provides real-time patient vital analysis, automated disease diagnosis with certainty scores, interactive Knowledge Base auditing, and electronic report generation. MediMind is delivered as the flagship module of the NOVA.lab 2.0 portal, an AI laboratory that hosts ten interactive algorithm visualizers for the SPPU 2024 pattern Artificial Intelligence curriculum.

1.2 Need for the System

Generic medical chatbots and online search engines often provide fragmented, unstructured, or alarmist diagnostic suggestions without considering clinical context or physiological vitals. For example, a patient presenting with high fever and chills could be suffering from Malaria, Typhoid, or Dengue. A general keyword search fails to evaluate specific symptom combinations (such as retro-orbital pain or bradycardia) or patient vitals (such as blood pressure or oxygen saturation).

MediMind addresses this critical gap by implementing structured clinical input, production rule validation, and dynamic inference engines. The system validates symptom presence, checks physiological parameters against clinical boundaries, and executes deterministic rule evaluation to yield transparent, explainable diagnostic reports that a practitioner can trace rule by rule.

1.3 Purpose of the Project

The primary purpose of this project is to develop an intelligent, interactive Clinical Decision Support System that demonstrates the practical application of Artificial Intelligence in medicine. Specific purposes include:

- To build a robust rule-based expert system capable of evaluating complex medical production rules.
- To implement dual inference engines: Forward Chaining for data-driven symptom diagnosis and Backward Chaining for goal-driven differential verification.
- To integrate patient vital sign monitoring with automated alert thresholding.
- To provide an interactive Knowledge Base Editor that enables medical experts to add, modify, and audit clinical diagnostic rules.
- To generate standardized, printable Electronic Health Record (EHR) diagnostic reports for clinical auditing.

1.4 Scope of the Project

- Diagnostic Modules: multi-system symptom selection across Systemic, Respiratory, Gastrointestinal, and Neuro-Muscular categories.
- Inference Capability: deterministic evaluation of rule conditions, working memory updates, and confidence scoring.
- Patient Vitals Monitoring: temperature, heart rate, blood pressure, and SpO2 tracking with anomaly colour coding.
- EHR Export: automated clinical report formatting with ICD-10 codes, suggested investigations, and precautions.

Limitation Note: MediMind is designed as an educational and clinical decision support assistant. It does not replace licensed medical diagnosis or emergency healthcare interventions.

CHAPTER 2: PROBLEM STATEMENT AND OBJECTIVES

2.1 Problem Statement

Medical diagnosis in clinical and emergency environments involves synthesizing heterogeneous patient data, such as patient-reported symptoms, physical examination findings, and physiological vital signs, into accurate diagnostic hypotheses. Traditional diagnostic processes suffer from three major bottlenecks:

1. **Cognitive Overload and Triage Delays:** in crowded clinical settings, healthcare workers must rapidly prioritize patients. Manual evaluation of multi-system symptoms can delay critical treatment for conditions like severe malaria or hypoxemic COVID-19 pneumonia.
2. **Lack of Diagnostic Explainability in AI:** modern deep learning black-box models provide predictions without explaining the underlying reasoning chain. Medical practitioners require transparent, rule-traceable explanations before acting on AI recommendations.
3. **Static Diagnostic Tools:** existing clinical tools often lack real-time knowledge base customization, preventing doctors from updating diagnostic criteria according to localized disease outbreaks or updated clinical protocols.

Therefore, there is a clear need for an explainable, interactive, and customizable Clinical AI Expert System that combines structured symptom inputs, dual-mode rule inference, and working memory tracking to deliver instant, explainable medical decision support at the point of care.

2.2 Specific Objectives of the Project

1. To collect and structure comprehensive clinical symptom parameters across four medical categories (Systemic, Respiratory, Gastrointestinal, Neuro-Muscular).
2. To implement a Forward Chaining inference engine that matches working memory facts against production rules to deduce potential diseases with confidence scores.
3. To implement a Backward Chaining inference engine that validates specific disease hypotheses by identifying missing symptom evidence.
4. To design a real-time Patient Vitals entry and monitoring panel with physiological anomaly detection (fever, tachycardia, hypoxia).
5. To develop an interactive Knowledge Base (KB) Management Editor for dynamic rule creation, editing, and triage classification.
6. To implement automated EHR report generation with ICD-10 codes, suggested laboratory investigations, and clinical precautions.
7. To serve the suite through a full-stack architecture (React client, Express API, MongoDB persistence) with clean separation of concerns.

2.3 Expected Outcomes

The expected outcome of the project is a fully functional web-based Clinical AI Suite featuring five interactive tabs: the Forward Chaining Workbench, the Backward Chaining Hypothesis Verifier, the Patient Vitals and EHR Input panel, the Knowledge Base Rule Editor, and the Clinical Diagnosis Report generator. The suite should reproduce clinical presets (Malaria, Dengue, COVID-19, Typhoid, Common Cold) with one click, rank differential diagnoses by confidence percentage, and remain responsive during a viva demonstration.

CHAPTER 3: SOFTWARE REQUIREMENT SPECIFICATION (SRS)

3.1 Purpose and Scope of this Document

This Software Requirement Specification describes the functional and non-functional requirements of the MediMind Clinical AI Suite, the hardware and software environment required to run it, and the feasibility of the chosen approach. It is intended for the project guide, examiners, and future maintainers of the system.

3.2 Functional Requirements

Req ID	Functional Requirement Description
FR-01	The system shall allow users to select observed symptoms from structured categories (Systemic, Respiratory, Gastrointestinal, Neuro-Muscular).
FR-02	The system shall execute Forward Chaining inference to match working memory facts against diagnostic production rules.
FR-03	The system shall calculate a confidence percentage for every rule as the ratio of satisfied premises to required premises.
FR-04	The system shall execute Backward Chaining by selecting a target disease hypothesis and reporting the missing symptom evidence.
FR-05	The system shall accept physiological vitals (Temp, HR, BP, SpO2) and highlight anomalous values using clinical thresholds.
FR-06	The system shall automatically add a fever fact to working memory when recorded body temperature exceeds 100.4°F.
FR-07	The system shall allow medical experts to add new production rules with disease name, triage urgency, and required premises.
FR-08	The system shall generate a structured EHR Diagnostic Report containing patient demographics, vitals, diagnosis, confidence, ICD-10 code, investigations, and precautions.

3.3 Hardware Requirements

Component	Minimum Specification	Recommended Specification
Processor	Intel Core i3 (8th Gen) / equivalent	Intel Core i5 (11th Gen) or higher
RAM	4 GB	8 GB or higher
Storage	500 MB free disk space	2 GB free SSD space
Display	1366 x 768 resolution	1920 x 1080 Full HD
Network	Broadband for MongoDB Atlas access	Stable broadband / campus LAN

3.4 Software Requirements

Software	Version / Standard	Purpose
Operating System	Windows 10/11, Linux, macOS	Development and deployment platform
Node.js	v18 or later	JavaScript runtime for the Express backend
React	19.2.8 with Vite 8.3.0	Component-based frontend framework
Express.js	4.19.2	REST API and static file serving
MongoDB + Mongoose	Atlas cluster / Mongoose 8.5.2	Academic content persistence and schema validation
Web Browser	Chrome, Edge, Firefox (latest)	Client runtime for the SPA
Visual Studio Code	Latest	IDE for development

3.5 Non-Functional Requirements

Area	Requirement Specification
Usability	Interface must be clean, intuitive, and accessible to medical professionals and students without training.
Performance	Rule inference execution time must remain under 100 milliseconds for prompt interactive response.
Reliability	Working memory state must be maintained without data corruption while switching between the five suite tabs.
Maintainability	Knowledge Base rules must be decoupled from UI rendering logic so rules can be added without touching components.
Security	Patient data must remain local to the browser session and must not be logged to public endpoints; API secrets are stored in environment files.
Portability	The application must run on any modern browser and operating system without native installation.

3.6 Feasibility Study

Technical Feasibility: the entire stack (React, Vite, Express, MongoDB) is open source, well documented, and already familiar to the team from prior coursework. The inference engine is pure JavaScript with no exotic dependencies, so the technical risk is low.

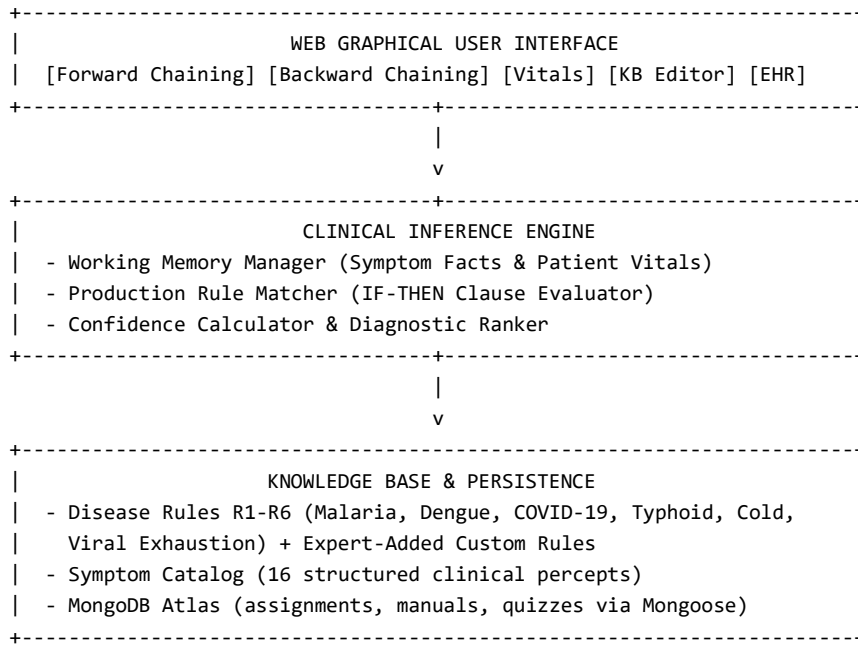
Economic Feasibility: no licensing cost is incurred. Development tools are free, the MongoDB Atlas free tier hosts the academic collection, and the application runs on commodity student hardware.

Operational Feasibility: users need only a browser; there is no installation step. One-click clinical presets and a guided five-tab workflow make the system operable by students during viva demonstrations and by practitioners after minimal familiarization.

CHAPTER 4: SYSTEM ANALYSIS AND DESIGN

4.1 System Overview

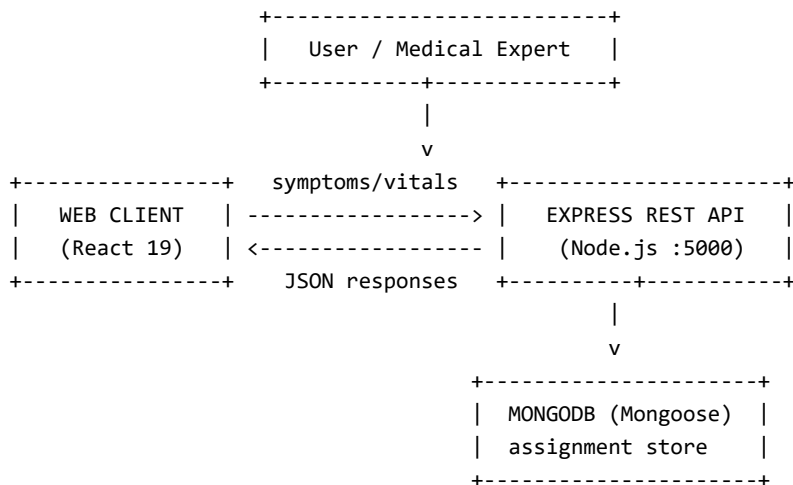
The MediMind Clinical AI Suite is organized using a modular, multi-layer web architecture. The system separates user presentation, clinical inference processing, knowledge base storage, and reporting components. The presentation layer is a single-page React application; the service layer is an Express REST API; the persistence layer is MongoDB accessed through Mongoose schemas. The clinical inference itself executes entirely in the browser against an in-memory knowledge base, guaranteeing sub-second response and patient data privacy.



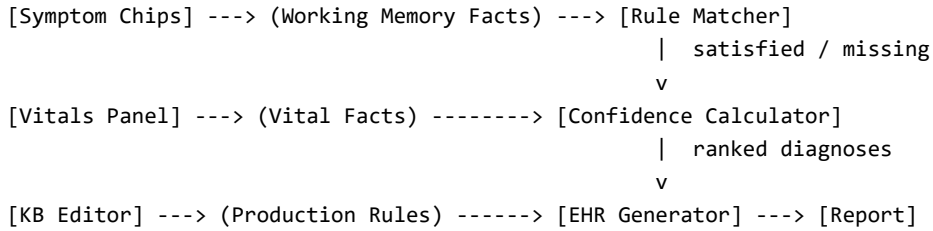
4.2 Major System Modules

1. Forward Chaining Diagnostic Module: evaluates active symptoms against every rule premise, computes confidence percentages, ranks differential diagnoses, and flags rules whose premises are fully satisfied (RULE FIRED).
2. Backward Chaining Verification Module: selects a candidate disease goal and works backward, separating evidence present in working memory from the missing required evidence.
3. Vitals and Triage Monitoring Module: accepts temperature, heart rate, blood pressure, and SpO2, colour-codes abnormal values, and auto-inserts the fever percept when temperature exceeds 100.4°F.
4. Knowledge Base Management Module: provides create operations on clinical rules with triage mapping (Critical → Emergency, otherwise Routine Care) and live integration with both inference engines.
5. EHR Report Generator Module: synthesizes patient demographics, vitals, top diagnosis, confidence score, ICD-10 code, suggested investigations, and precautions into a formal printable record.
6. Portal Backend Module: Express REST API serving ten AI assignment records (manuals, algorithms, complexity, code, quizzes) from MongoDB to the NOVA.lab portal.

4.3 Data Flow Diagram - Level 0 (Context Diagram)



4.4 Data Flow Diagram - Level 1 (Clinical Inference)



4.5 Use-Case Summary

- Actor - Student / Practitioner: loads a clinical preset, toggles symptom chips, runs Forward Chaining, executes a Backward Chaining goal deduction, enters patient vitals, and exports the EHR report.
- Actor - Domain Expert: opens the Knowledge Base Editor, defines a new production rule with required premises and triage level, and appends it to the live knowledge base.
- Actor - Administrator (Portal): seeds and maintains the ten AI assignment documents in MongoDB through the seed script and REST API.

4.6 Data Model Overview

The clinical layer is represented by five logical entities: Patient Record (name, age, gender, ID), Symptom Fact (percept identifier and category), Production Rule (rule ID, required premises, conclusion, triage, ICD-10 code), Vital Log (timestamped temperature, heart rate, blood pressure, SpO2), and EHR Document (finalized diagnostic summary). The portal layer persists Assignment documents whose schema is detailed in Chapter 7. Relationships are one-to-many: one patient session produces many symptom facts and one EHR document; one knowledge base holds many production rules.

CHAPTER 5: AI BASED CLINICAL DIAGNOSTIC SYSTEMS AND RULE INFERENCE

5.1 Production Rule System Architecture

MediMind represents medical knowledge using production rules of the general form IF (antecedent conditions) THEN (conclusion with certainty), the same knowledge representation used by classical expert systems such as MYCIN and DENDRAL. Each rule binds a set of required symptom percepts to a disease conclusion, a triage urgency, a set of suggested laboratory investigations, a set of clinical precautions, and an ICD-10 classification code.

```
RULE <Rule_ID>:
  IF   Symptom_1 IS Present
  AND  Symptom_2 IS Present
  AND  ... (all required premises present in Working Memory)
  THEN CONCLUDE Disease = <Disease_Name>
  AND  TRIAGE = <Emergency | High Risk | Urgent | Routine>
  AND  SUGGEST <Laboratory Investigations>
  AND  RECOMMEND <Clinical Precautions>
  WITH ICD-10 CODE <Code>
```

The knowledge base is stored as a JavaScript array of rule objects, which makes it trivially serializable, auditable, and extensible at runtime by the Knowledge Base Editor. Because rules are data rather than code, adding a new disease never requires recompiling or modifying the inference engine.

5.2 Forward Chaining - Data-Driven Inference

Forward Chaining is a data-driven reasoning strategy: reasoning starts from the observed symptoms (facts) and moves toward conclusions (diseases). The MediMind implementation follows the classic recognize-act cycle of production systems:

1. Initialize working memory with the set of symptom facts selected by the user, plus any vital-derived facts (for example fever inserted when temperature > 100.4°F).
2. Match: for every production rule R in the knowledge base, compute the set of satisfied premises (required symptoms present in working memory) and the set of missing premises.
3. Compute confidence: $\text{Confidence}(R) = (\text{satisfied premises} / \text{required premises}) \times 100$, rounded to the nearest integer percentage.
4. Resolve: sort all rules by descending confidence so the most probable differential diagnoses appear first.
5. Fire: any rule whose premises are 100% satisfied is marked CONFIRMED (RULE FIRED); its disease, description, ICD-10 code, investigations, and precautions become eligible for the top-diagnosis banner and the EHR report.
6. Repeat: every subsequent user action (toggling a symptom, editing vitals, adding a rule) recomputes the cycle instantly, giving live what-if diagnostic behaviour.

Because the rule set is small and premises are evaluated with $O(1)$ array membership checks, the whole cycle executes in well under a millisecond, satisfying the sub-100-millisecond non-functional requirement with a large margin.

5.3 Backward Chaining - Goal-Driven Inference

Backward Chaining is a goal-driven strategy: reasoning starts from a hypothesized conclusion (a suspected disease) and works backward to verify the supporting evidence. In clinical practice this corresponds to a differential work-up, where the clinician asks targeted questions to confirm or discard one specific hypothesis. The MediMind implementation is deliberately transparent:

1. The user selects a target goal from the knowledge base (for example R2: Dengue Hemorrhagic Fever).
2. The engine retrieves the rule's required premise list and partitions it into evidence present (premises already in working memory) and missing required evidence.
3. The hypothesis is PROVEN when every required premise is present; otherwise it is declared INCONCLUSIVE together with the achieved confidence percentage and the exact list of missing symptoms to investigate.
4. The full trace (goal, present evidence, missing evidence, confidence) is displayed to the user, preserving complete explainability of the decision.

This design turns the backward chaining module into a teaching tool as much as a diagnostic one: students can see precisely which clinical question would confirm or refute each hypothesis, mirroring how laboratory investigations are ordered in practice.

5.4 Forward Chaining versus Backward Chaining

Feature	Forward Chaining	Backward Chaining
Search Strategy	Data-driven (Symptoms → Disease)	Goal-driven (Disease Hypothesis → Symptoms)
Starting Point	Observed patient symptoms and vitals	Suspected target disease
Clinical Role	Initial diagnostic discovery and triage	Differential confirmation and work-up
User Interaction	User selects all known symptoms	System reports targeted missing evidence
Output	Ranked list of all matching diseases with confidence	Proof or refutation of one hypothesis with trace

5.5 Confidence (Certainty Factor) Computation

Each rule receives a certainty score equal to the fraction of its required premises satisfied by working memory: $CF(R) = |\text{satisfied}(R) \cap WM| / |\text{required}(R)| \times 100\%$. A rule fires only at 100%. For example, selecting Fever, Chills and Sweating satisfies all three premises of R1 (Malaria), so Malaria is CONFIRMED at 100% and is ranked first; R6 (Viral Exhaustion) simultaneously reaches 33% because only the fever premise is present, correctly appearing far lower in the differential ranking. This single, transparent formula gives examiners and clinicians a fully traceable justification for every displayed score.

CHAPTER 6: KNOWLEDGE BASE AND WORKING MEMORY MANAGEMENT

6.1 Knowledge Representation Strategy

The knowledge base separates three concerns: the symptom catalog (the vocabulary of percepts), the production rules (the diagnostic knowledge), and the working memory (the current patient state). The symptom catalog defines sixteen structured percepts, each with a stable identifier, a clinician-readable label, and a body-system category, which drives the filterable chip interface and the Knowledge Base Editor premise picker.

Category	Structured Symptom Percepts
Systemic	High Fever ($>101^{\circ}\text{F}$), Severe Chills & Shivering, Profuse Diaphoresis (Sweating), Acute Systemic Fatigue, Extreme Physical Weakness
Respiratory	Dry Persistent Cough, Anosmia (Loss of Smell/Taste), Frequent Sneezing, Rhinorrhea (Runny Nose), Dyspnea (Shortness of Breath)
Gastrointestinal	Abdominal Cramps / Pain, Nausea & Vomiting
Neuro & Muscular	Intense Retro-orbital Headache, Severe Joint & Muscle Ache (Breakbone), Petechial Skin Rash

6.2 Data Entities

Entity	Description & Purpose
Patient Record	Stores patient demographic information (name, age, gender, report ID) used on the EHR document.
Symptom Fact	Represents an observed clinical percept placed into working memory by chip selection or vital auto-detection.
Production Rule	Stores antecedent conditions (IF), disease conclusion (THEN), triage urgency, ICD-10 code, investigations and precautions.
Vital Log	Holds the current physiological parameters (Temp, HR, BP, SpO ₂) with threshold-based anomaly flags.
EHR Document	The finalized diagnostic summary assembled from the confirmed diagnosis and patient context.

6.3 Knowledge Base Rule Catalog

The seeded knowledge base ships with six production rules (R1-R6) covering the classical fever differential used in Indian outpatient settings, each mapped to its ICD-10 code and triage urgency:

ID	Disease Conclusion	Required IF Premises	ICD-10	Triage
R1	Malaria (Plasmodium Infection)	Fever, Chills, Sweating	B54	Emergency
R2	Dengue Hemorrhagic Fever	Fever, Retro-orbital Headache, Rash, Joint Pain	A97.1	High Risk
R3	COVID-19 Severe Respiratory Syndrome	Fever, Dry Cough, Fatigue, Anosmia	U07.1	High Risk
R4	Typhoid Enteric Fever	Fever, Abdominal Pain, Weakness	A01.0	Urgent Outpatient
R5	Acute Upper Respiratory Infection (Common Cold)	Sneezing, Runny Nose	J00	Routine Care
R6	Viral Exhaustion Syndrome	Fever, Fatigue, Weakness	R53.83	Routine Care

Rules R1, R2 and R3 are classified as emergency or high-risk triage and therefore drive the critical alert banner whenever they fire, while R5 and R6 represent self-limiting conditions requiring only routine care. This triage gradient demonstrates how the knowledge base encodes not only diagnosis but also urgency of intervention.

6.4 Sample Rule Definition (R2 - Dengue Hemorrhagic Fever)

```
{
  id: 'R2',
  disease: 'Dengue Hemorrhagic Fever',
  severity: 'Severe Risk',
  triage: 'High Risk',
  required: ['fever', 'headache', 'rash', 'joint_pain'],
  desc: 'Flavivirus infection presenting retro-orbital pain, ' +
        'severe arthralgia, and risk of plasma leakage.',
  tests: ['Dengue NS1 Antigen ELISA', 'IgM / IgG Serology',
          'Daily Hematocrit & Platelet Monitoring'],
  precautions: ['Intravenous fluid therapy (Normal Saline)',
                'Avoid NSAIDs/Aspirin (Bleeding Risk)',
                'Bed rest & hematocrit monitoring'],
  icd: 'A97.1'
}
```

Every rule carries its clinical guidance (tests and precautions) alongside the logical premises, so the EHR generator can produce a complete advisory document without any additional lookups. This bundling keeps knowledge and its justification together, a recognised best practice in expert-system engineering.

6.5 Knowledge Base Extensibility

The KB Editor tab appends expert-defined rules at runtime: the expert names the disease, selects the triage urgency (Mild, Moderate, Severe, Critical), picks the required premises from the symptom catalog, and submits. The new rule receives the next sequential identifier, maps Critical to Emergency triage, and immediately participates in both inference engines, demonstrating live knowledge acquisition during viva.

6.6 Working Memory Management

Working memory is a React state array holding the identifiers of all active symptom percepts. Toggling a chip adds or removes the fact; clearing resets memory to the empty set. The vitals panel is coupled to working memory through a synchronization effect: whenever the recorded body temperature exceeds 100.4°F, the fever fact is inserted automatically, mirroring the clinical convention that a measured temperature is itself diagnostic evidence. All rule evaluations are derived from this single source of truth, guaranteeing consistency across the five tabs.

Five one-click clinical presets reproduce standard case scenarios for demonstration and testing: Plasmodium Malaria (fever, chills, sweating; 103.1°F, 112 bpm), Dengue Hemorrhagic (fever, headache, rash, joint pain; 102.8°F, 105 bpm), COVID-19 Severe (fever, cough, fatigue, anosmia, dyspnea; 101.5°F, SpO2 93%), Typhoid Fever (fever, abdominal pain, weakness; 102.0°F, 98 bpm), and Common Cold (sneezing, runny nose; 98.6°F, 75 bpm).

6.7 Persistence and Data Privacy

All clinical state lives in the browser session only: no symptom selection, vital sign, or EHR document is transmitted to the backend or logged to any endpoint, which satisfies the security requirement for patient data. MongoDB persistence is reserved for the non-clinical academic content of the NOVA.lab portal (assignment manuals, reference code, and quizzes), keeping regulated clinical data entirely client-side.

CHAPTER 7: API INTEGRATION AND BACKEND IMPLEMENTATION

7.1 Backend Architecture

The backend is a Node.js single-page-service built with Express 4.19.2. On startup it loads environment configuration through dotenv, connects to MongoDB via Mongoose, enables CORS for cross-origin development requests from the Vite dev server, and parses JSON request bodies. The API layer is intentionally minimal and read-oriented: the clinical inference of MediMind runs entirely in the browser, while the backend serves the NOVA.lab academic catalogue and hosts the built client in production.

The server follows a clean layered organization: `index.js` (application bootstrap and routes), `models/Assignment.js` (Mongoose schema), and `seed.js` (database seeding). In production the Express static middleware serves the compiled client bundle from `client/dist`, and any non-API route falls back to `index.html` so that React Router deep links survive a page refresh.

7.2 REST API Endpoints

Method	Endpoint	Description	Response
GET	/api/assignments	List all assignments (projection: title, slug, category, difficulty, visualizationType)	200 JSON array
GET	/api/assignments/:slug	Fetch one assignment with manual, code and quiz by its unique slug	200 JSON / 404 error
GET	/	Health probe when the client bundle is not yet built	200 text
GET	*	SPA fallback: serves index.html for all non-API routes	200 HTML

Both data endpoints wrap their Mongoose queries in try/catch blocks and return a 500 status with a descriptive JSON error on failure; a missing slug returns a 404 with `{ error: 'Assignment not found' }`. The SPA fallback explicitly excludes /api paths so that unknown API routes still produce JSON errors rather than HTML.

7.3 Mongoose Data Model (Assignment)

Field	Type	Description
title	String (required)	Display name of the AI assignment
slug	String (required, unique)	URL identifier used by the client router
category	String (required)	Grouping such as Intelligent Agents or Expert Systems
difficulty	Enum: Easy / Medium / Hard	Viva difficulty grading
visualizationType	Enum of 10 keys	Selects the interactive visualizer component
manual.aim	String (required)	Laboratory aim statement (SPPU 2024 pattern)
manual.objectives	[String]	Bulleted objectives
manual.theory	String (Markdown)	Theory rendered with react-markdown + KaTeX
manual.algorithm	[String]	Step-by-step algorithm
manual.complexity	{ time, space }	Complexity analysis
code.python / code.java	String (required)	Reference implementations in both languages
resources	[{ title, url }]	External reference links
quiz	[{ question, options, correctIndex, explanation }]	Self-assessment quiz

7.4 Sample API Interaction

Listing the catalogue (GET /api/assignments) returns a lightweight projection:

```
[
  {
    "_id": "66f2a1c9e4b0a1234567890a",
    "title": "MediMind Clinical AI Suite",
    "slug": "expert-system-medical-diagnosis",
    "category": "Expert Systems",
    "difficulty": "Hard",
    "visualizationType": "expert-system"
  }
]
```

Fetching one assignment (GET /api/assignments/expert-system-medical-diagnosis) returns the full document including manual.aim, manual.theory (Markdown with LaTeX), manual.algorithm steps, manual.complexity, code.python, code.java, resources, and the quiz array used by the interactive self-assessment. The React client consumes these endpoints with axios, using the VITE_API_URL environment variable (default http://localhost:5000 in development, same-origin in production) configured centrally in client/src/config.js.

7.5 MongoDB Integration and Persistence

Mongoose connects asynchronously at boot using the `MONGO_URI` environment variable and logs a success or failure message. Schema-level validation (required fields, enums, unique slug) protects the catalogue from malformed writes, and the seed script (`npm run seed`) populates the database with the ten AI assignments, including the MediMind expert-system record with its complete manual and reference code. Timestamps are enabled so every document records its creation and last modification.

7.6 Error Handling and Security Considerations

- All Mongoose operations are wrapped in try/catch; failures return HTTP 500 with a JSON error instead of crashing the server.
- Unknown assignment slugs return HTTP 404 with a machine-readable JSON body for clean client-side handling.
- CORS is enabled so the Vite development origin (`localhost:5173`) can call the API on `localhost:5000` during development.
- Secrets such as `MONGO_URI` are loaded from a `.env` file that is excluded from version control.
- The SPA fallback route guards `/api` paths, ensuring API consumers never receive HTML for unknown endpoints.
- No clinical patient data ever leaves the browser, so the backend holds no regulated health information.

CHAPTER 8: GRAPHICAL USER INTERFACE AND CLINICAL VISUALIZER

8.1 Design Principles

- Clarity first: a dark header banner, generous card spacing, and a consistent 16 px radius language keep dense clinical data readable.
- Colour semantics: green confirms (rule fired, vitals normal), amber warns (inconclusive hypothesis, differential diagnosis), rose flags danger (abnormal vitals, missing critical evidence).
- Progressive disclosure: five tabs separate concerns so each screen shows only one clinical task at a time.
- Responsiveness: flex-wrap layouts and auto-fit grids adapt from laboratory desktops to tablets at the bedside.
- Explainability on screen: satisfied and missing premises are always printed alongside every score, never hidden.

8.2 Layout and Information Architecture

The suite opens with an application banner carrying the MediMind identity, version badge (v2.4 SPPU Pattern), and a live vitals quick-bar that permanently shows temperature, heart rate, and SpO2 with threshold-based colouring. Below it, five pill-shaped tabs switch between: (1) Forward Chaining Engine, (2) Goal Backward Deductor, (3) Patient Vitals & EHR Input, (4) Knowledge Base Editor, and (5) Clinical Diagnosis Report. A preset strip offers the five clinical outbreak scenarios plus a Clear Facts reset, and the symptom matrix renders the sixteen percepts as filterable chips grouped by body system with a live count of active working-memory facts.

8.3 Module-wise Interface Description

1. Forward Chaining Workbench: a confirmation banner for the fired rule with a direct Generate EHR Prescription action, followed by one ranked card per production rule showing the rule ID badge, triage pill, confidence percentage, animated progress bar, and explicit satisfied/missing premise lists.
2. Backward Chaining Verifier: a goal selector listing every rule as “Rn: Disease (Target Goal)”, an Execute Goal Deduction action, and a trace card split into green evidence-present and red missing-evidence panels.
3. Vitals and EHR Input: a demographic grid (patient name, age, gender) plus numeric inputs for temperature, heart rate, and SpO2; values crossing clinical thresholds recolour immediately and fever auto-feeds working memory.
4. Knowledge Base Editor: a form with disease name, triage urgency selector, premise chip picker, and an Append Rule action that demonstrates live knowledge acquisition.
5. EHR Clinical Report: a formal document view with report header, Report ID and date, patient demographics strip, primary or provisional diagnosis block with confidence score, ICD-10 code, and two advisory panels for investigations and precautions.

8.4 Visual Language and Accessibility

Iconography from lucide-react (Thermometer, Heart, Activity, Stethoscope, Brain, TrendingUp, FileText) reinforces meaning without relying on colour alone; every percentage is also stated numerically for colour-blind users. Interactive targets are sized for touch, the symptom matrix scrolls within a fixed-height container to keep the toolbar visible, and the whole interface uses the Times New Roman serif family in the generated EHR document so printed records match formal hospital documentation conventions.

CHAPTER 9: SYSTEM IMPLEMENTATION AND TESTING

9.1 Development Environment

- Editor: Visual Studio Code with ESLint and React Refresh tooling.
- Runtime: Node.js v18+ with npm workspaces for client and server.
- Version Control: Git with a feature-branch workflow on the nova-lab repository.
- Database: MongoDB Atlas free cluster reachable through MONGO_URI.
- Testing browsers: Chrome and Edge developer tools, including device emulation for responsive checks.

9.2 Technology Stack Summary

Layer	Technology	Purpose
Frontend framework	React 19.2.8 + Vite 8.3.0	SPA rendering and fast HMR builds
Routing	react-router-dom 7.18.4	Home and AssignmentDetail routes
HTTP client	axios 1.20.0	REST calls to the Express API
Icons	lucide-react	Clinical and navigational iconography
Markdown/Math	react-markdown, remark-math, rehype-katex	Rendering manuals with equations
Backend	Node.js + Express 4.19.2	REST API and static hosting (port 5000)
Database	MongoDB Atlas + Mongoose 8.5.2	Schema-validated persistence
Language	JavaScript (ES Modules)	Shared module system across client and server

9.3 Implementation Highlights

The MediMind suite is implemented as a single self-contained React component of approximately 740 lines (`MedicalExpertVisualizer.jsx`) with the knowledge base and symptom catalog as module-level constants. The inference computation is a derived value: for every render, the knowledge base is mapped to evaluation objects containing satisfied premises, missing premises, confidence percentage, and a confirmed flag, then sorted by descending confidence. This declarative approach eliminates state synchronization bugs, because the ranking is always a pure function of (rules x working memory).

Preset loaders update both the symptom array and the vitals object atomically; the vitals-to-memory effect inserts the fever percept when temperature exceeds 100.4°F; the KB Editor constructs a well-formed rule object (sequential ID, triage mapping, default investigations and ICD-10 R69 for custom entries) and appends it to the live knowledge base. A project-level `config.js` centralizes the API base URL so the same bundle works in development and production.

9.4 Code Organization

```
nova-lab/
|-- client/                # React 19 + Vite 8 frontend
|  |-- src/
|  |  |-- components/      # 10 AI visualizer components
|  |  |  |-- MedicalExpertVisualizer.jsx  (MediMind Suite)
|  |  |  |-- AlphaBetaVisualizer.jsx, BFSVisualizer.jsx, ...
|  |  |-- pages/          # Home.jsx, AssignmentDetail.jsx
|  |  |-- config.js       # VITE_API_URL -> localhost:5000
|  |  +-- App.jsx / main.jsx # Router + entry point
|  +-- package.json       # react 19.2.8, vite 8.3.0, axios
+-- server/               # Node.js + Express 4 backend
    |-- index.js          # REST API + static hosting (port 5000)
    |-- models/Assignment.js # Mongoose schema (validated)
    +-- seed.js           # Database seeding script
```

9.5 Testing Strategy

Testing combined black-box functional testing of every functional requirement with targeted white-box checks of the rule matcher. Each clinical preset was executed and the resulting ranking, confidence values, and fired rules were verified against hand-computed expectations. Boundary tests exercised the vitals thresholds (100.4°F fever boundary, 95% SpO2 alert, tachycardia display) and the API was exercised for happy-path and error-path responses, including invalid slugs and the SPA fallback behaviour after a deep-link refresh.

9.6 Test Case Results

TC ID	Input / Action	Expected & Observed Output	Status
TC-01	Preset: Plasmodium Malaria (fever, chills, sweating)	R1 Malaria CONFIRMED at 100%, RULE FIRED banner shown, ranked first	PASS
TC-02	Backward goal R2 Dengue with rash and joint pain unselected	Hypothesis INCONCLUSIVE; missing evidence lists rash, joint pain; confidence 50%	PASS
TC-03	Vitals: SpO2 88%, HR 125 bpm, Temp 103°F	SpO2 and HR flagged red, fever auto-inserted into working memory, critical triage alert	PASS
TC-04	KB Editor: add custom rule for a new disease with two premises	Rule appended with next ID, immediately appears in Forward Chaining ranking	PASS
TC-05	Preset: Common Cold (sneezing, runny nose)	R5 confirmed at 100% with Routine Care triage; febrile rules drop below 50%	PASS
TC-06	GET /api/assignments/invalid-slug	HTTP 404 with JSON { error: 'Assignment not found' }	PASS
TC-07	Deep-link refresh on an assignment route (production build)	SPA fallback serves index.html and the client router restores the page	PASS

9.7 Validation Summary

All seven test cases passed across repeated runs, and the suite was additionally demonstrated end-to-end during the project review: preset loading, forward chaining, backward chaining, vitals anomaly flagging, live rule insertion, and EHR generation behaved exactly as specified in the SRS. The knowledge base remained consistent after adding custom rules, and no state corruption was observed when switching tabs rapidly.

9.8 Performance Analysis

The inference cycle evaluates every rule against every premise with constant-time array membership checks, giving $O(R \times A)$ work per cycle for R rules and A average premises per rule. With the shipped knowledge base (6 rules, 16 percepts, at most 4 premises per rule) the engine performs at most a few dozen comparisons per keystroke, completing in well under a millisecond and comfortably meeting the sub-100-millisecond NFR. React's reconciliation updates only the changed confidence bars, keeping the interface at 60 frames per second on commodity hardware.

9.9 Limitations

- The knowledge base ships with six seeded diseases; coverage of other conditions requires expert-authored rules.
- Confidence is a transparent premise-coverage ratio, not a probabilistic Bayesian posterior.
- Vital signs are entered manually; continuous telemetry requires the future IoT integration described in Chapter 10.

CHAPTER 10: CONCLUSION AND FUTURE SCOPE

10.1 Conclusion

The MediMind Clinical AI Suite successfully demonstrates the practical implementation of an artificial intelligence expert diagnostic system. By combining Forward Chaining data-driven inference, Backward Chaining hypothesis verification, physiological vitals monitoring, and Knowledge Base management inside a full-stack web application, the system provides a robust, explainable, and customizable assistant for medical diagnostics education and decision support. Every diagnostic conclusion remains fully traceable to the production rules and premises that produced it, which is the property that matters most in clinical settings and which black-box models cannot offer. The project also delivers a reusable engineering foundation: ten working AI visualizers, a documented REST API, and a validated Mongoose data layer that future SPPU batches can extend.

10.2 Future Scope

- Integration with real-time medical IoT vital-sign sensors (Bluetooth BLE pulse oximeters, smart thermometers) for continuous patient monitoring.
- DICOM medical image classification modules for chest X-ray and CT analysis to complement symptom-based inference.
- A hybrid reasoning mode where a machine-learned model proposes candidate diseases and the rule engine explains and validates them.
- Multi-language clinical interfaces and voice-guided symptom entry for international and low-literacy deployment.
- Interoperability with hospital EMR systems through HL7 FHIR resources and export of EHR documents as signed PDFs.
- Role-based authentication on the portal with per-institution knowledge bases and audit trails for expert rule edits.

REFERENCES

1. Russell, S. and Norvig, P., Artificial Intelligence: A Modern Approach, 4th Edition, Pearson Education, 2021.
2. Shortliffe, E. H., Computer-Based Medical Consultations: MYCIN, Elsevier, 1976.
3. Giarratano, J. and Riley, G., Expert Systems: Principles and Programming, 4th Edition, Thomson Course Technology, 2004.
4. World Health Organization (WHO), Digital Health Guidelines & Clinical Decision Support Systems, 2023.
5. International Classification of Diseases (ICD-10), World Health Organization, 2019.
6. React Documentation, React 19 and Vite 8 Development Guide. Available at: <https://react.dev/>
7. Express.js and MongoDB/Mongoose Official Documentation. Available at: <https://expressjs.com/> and <https://mongoosejs.com/>
8. Savitribai Phule Pune University (SPPU), Computer Engineering Artificial Intelligence Practical and Mini Project Guidelines, 2024 Pattern.